

LAB 8

1. Write the programme to open a text file named input 2, and copy its contents to an output text file output 2.

PROGRAM:

```
package Lab8;

import java.io.*;

public class InputFileCopyDemo {

    public static void main(String[] args) {
        // Define input and output file paths using File objects
        File inputFile = new File("D:\\Notes\\Anudip\\input2.txt");
        File outputFile = new File("D:\\Notes\\Anudip\\output2.txt");

        // Initialize FileReader and FileWriter to null for later use
        FileReader reader = null;
        FileWriter writer = null;

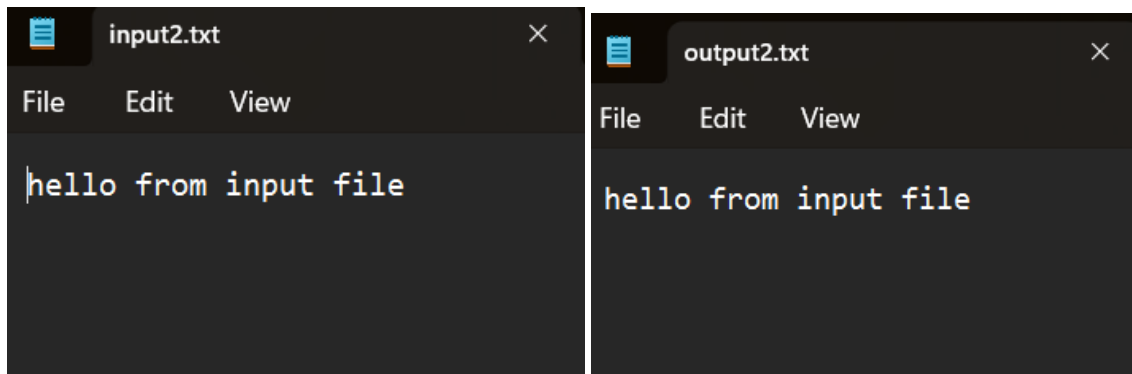
        try {
            // Create FileReader for reading from input file
            reader = new FileReader(inputFile);
            // Create FileWriter for writing to output file
            writer = new FileWriter(outputFile);

            int character;
            // Loop to read each character from input file until end-of-file (-1)
            while ((character = reader.read()) != -1) {
                // Write the character to the output file
                writer.write(character);
            }

            // Print confirmation message after successful file copy
            System.out.println("File copied successfully!");
        } catch (IOException e) {
            // Handle any IO exceptions during file operations
            System.out.println("An error occurred: " + e.getMessage());
        } finally {
            try {
                // Ensure FileReader is closed if it was opened
                if (reader != null) {
                    reader.close();
                }
                // Ensure FileWriter is closed if it was opened
                if (writer != null) {
                    writer.close();
                }
            } catch (IOException e) {
                // Handle exceptions that might occur during the closing of files
                System.out.println("An error occurred while closing the files: " +
e.getMessage());
            }
        }
    }
}
```

OUTPUT:

```
<terminated> InputFileCopyDemo [Java Application] C:\
File copied successfully!
```



2. Write the programme to show multithreading for the string "multi threads". Show the resulting output.

PROGRAM:

```
package Lab8;

class SimpleThread extends Thread {
    private char character;

    // Constructor to assign character to this thread
    public SimpleThread(char character) {
        this.character = character;
    }

    @Override
    public void run() {
        // Print the character
        System.out.print(character + " ");
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        String text = "multi threads";

        // Loop through each character in the string
        for (int i = 0; i < text.length(); i++) {
            // Create a new thread for the current character
            SimpleThread thread = new SimpleThread(text.charAt(i));
            thread.start(); // Start the thread

            try {
                // Introduce a short delay based on the index
                Thread.sleep(100 * i);
                // Wait for the current thread to finish before starting the next
                thread.join();
            } catch (InterruptedException e) {
```

```

        System.out.println("Thread interrupted: " + e.getMessage());
    }
}
}
}
}

```

OUTPUT:

```

<terminated> MultiThreadDemo [Java Application] C:\User
m u l t i   t h r e a d s

```

3. Implement a Java program that creates a thread using the Runnable interface. The thread should print numbers from 1 to 10 with a delay of 1 second between each number.

PROGRAM:

```

package Lab8;

public class RunnableInterfaceDemo implements Runnable {

    @Override
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println(i);
            try {
                // Pause the thread for 1 second (1000 milliseconds)
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted: " + e.getMessage());
            }
        }
    }

    public static void main(String[] args) {

        RunnableInterfaceDemo R1 = new RunnableInterfaceDemo();

        Thread t1 = new Thread(R1);
        t1.start();

    }
}

```

OUTPUT:

<terminated> RunnableInterfaceDemo [Java Applicat

1
2
3
4
5
6
7
8
9
10

4. Write a Java program that creates and starts three threads. Each thread should print its name and count from 1 to 5 with a delay of 500 milliseconds between each count.

PROGRAM:

```
package Lab8;

public class ThreeThreadsDemo implements Runnable {

    private String threadName;

    // Constructor to set the thread name
    public ThreeThreadsDemo(String threadName) {
        this.threadName = threadName;
    }

    @Override
    public void run() {
        // Print numbers from 1 to 5 with a delay
        for (int i = 1; i <= 5; i++) {
            System.out.println(threadName + ": " + i);
            try {
                // Wait for 500 milliseconds
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(threadName + " interrupted: " +
e.getMessage());
            }
        }
    }

    public static void main(String[] args) {
        // Create instances of ThreeThreadsDemo with different names
        ThreeThreadsDemo D1 = new ThreeThreadsDemo("Thread 1");
        ThreeThreadsDemo D2 = new ThreeThreadsDemo("Thread 2");
        ThreeThreadsDemo D3 = new ThreeThreadsDemo("Thread 3");

        // Create threads for each instance
        Thread T1 = new Thread(D1);
        Thread T2 = new Thread(D2);
        Thread T3 = new Thread(D3);

        // Start the threads
        T1.start();
```

```

        T2.start();
        T3.start();
    }
}

```

OUTPUT:

```

<terminated> Thread3L
Thread 3: 1
Thread 1: 1
Thread 2: 1
Thread 2: 2
Thread 3: 2
Thread 1: 2
Thread 3: 3
Thread 1: 3
Thread 2: 3
Thread 3: 4
Thread 1: 4
Thread 2: 4
Thread 3: 5
Thread 1: 5
Thread 2: 5

```

5. Create a Java program that demonstrates thread priorities. Create three threads with different priorities and observe the order in which they execute.

PROGRAM:

```

package Lab8;

public class ThreadPriorityDemo implements Runnable {

    private String threadName;

    // Constructor to set the thread's name
    public ThreadPriorityDemo(String threadName) {
        this.threadName = threadName;
    }

    @Override
    public void run() {

        // Print numbers with a delay
        for (int i = 1; i <= 5; i++) {
            System.out.println(threadName + ": " + i);
            try {
                Thread.sleep(500); // Wait for 500 milliseconds
            } catch (InterruptedException e) {
                System.out.println(threadName + " interrupted: " +
e.getMessage());
            }
        }
    }
}

```

```

    }

    public static void main(String[] args) {
        // Create Runnable tasks with different names
        Runnable task1 = new ThreadPriorityDemo("Low Priority Thread");
        Runnable task2 = new ThreadPriorityDemo("Medium Priority Thread");
        Runnable task3 = new ThreadPriorityDemo("High Priority Thread");

        // Create threads with different priorities
        Thread thread1 = new Thread(task1);
        Thread thread2 = new Thread(task2);
        Thread thread3 = new Thread(task3);

        // Set priorities
        thread1.setPriority(Thread.MIN_PRIORITY); // Lowest priority
        thread2.setPriority(Thread.NORM_PRIORITY); // Default priority
        thread3.setPriority(Thread.MAX_PRIORITY); // Highest priority

        // Start threads
        thread1.start();
        thread2.start();
        thread3.start();
    }
}

```

OUTPUT:

```

Terminated: ThreadPriorityDemo.java Application Core.
High Priority Thread: 1
Medium Priority Thread: 1
Low Priority Thread: 1
High Priority Thread: 2
Low Priority Thread: 2
Medium Priority Thread: 2
High Priority Thread: 3
Medium Priority Thread: 3
Low Priority Thread: 3
High Priority Thread: 4
Medium Priority Thread: 4
Low Priority Thread: 4
High Priority Thread: 5
Medium Priority Thread: 5
Low Priority Thread: 5

```